FUNDAMENTALS OF REINFORCEMENT LEARNING — TASK 2 LABORATORY ASSESSMENT

FLOWPICK: ADAPTIVE WAREHOUSE LEARNING AGENT USING TABULAR Q-LEARNING

Zian Rajeshkumar Surani (RA2311026050085) | **Vaishnavi Soni** (RA2311026050070)
Department of Computer Science and Engineering | SRM Institute of Science and Technology

Executive Abstract

Modern order fulfillment facilities rely increasingly on Automated Guided Vehicles (AGVs) capable of self-directed corridor navigation without brittle pre-programmed trajectory scripts. This academic report presents FlowPick, a model-free Reinforcement Learning (RL) system driven by discrete tabular Q-Learning. Operating inside a discrete 2D warehouse grid with static storage rack obstacles, the autonomous agent iteratively discovers an optimal routing policy to collect inventory items from a pickup depot and deliver them to a dispatch terminal. Evaluated across extensive episodes, the system demonstrates empirical policy convergence, asymptotic reduction in step latency, and robust transition from unguided exploration to a deterministic optimal path achieving a 100% greedy delivery success rate.

1. Introduction & Operational Motivation

Intralogistics environments are inherently constrained: storage racks limit traversable space, corridor bottlenecks produce congestion, and inventory transport demands rapid, cost-optimal paths. Traditional search heuristics (such as static A* and Dijkstra algorithms) compute geometric shortest routes but require complete prior environmental graph knowledge and cannot naturally optimize parameterized multi-objective rewards. Reinforcement Learning re-frames warehouse dispatching as an autonomous Markovian trial-and-error game where the agent observes sensory states, selects directional actions, and updates expected future payoffs via scalar environmental rewards.

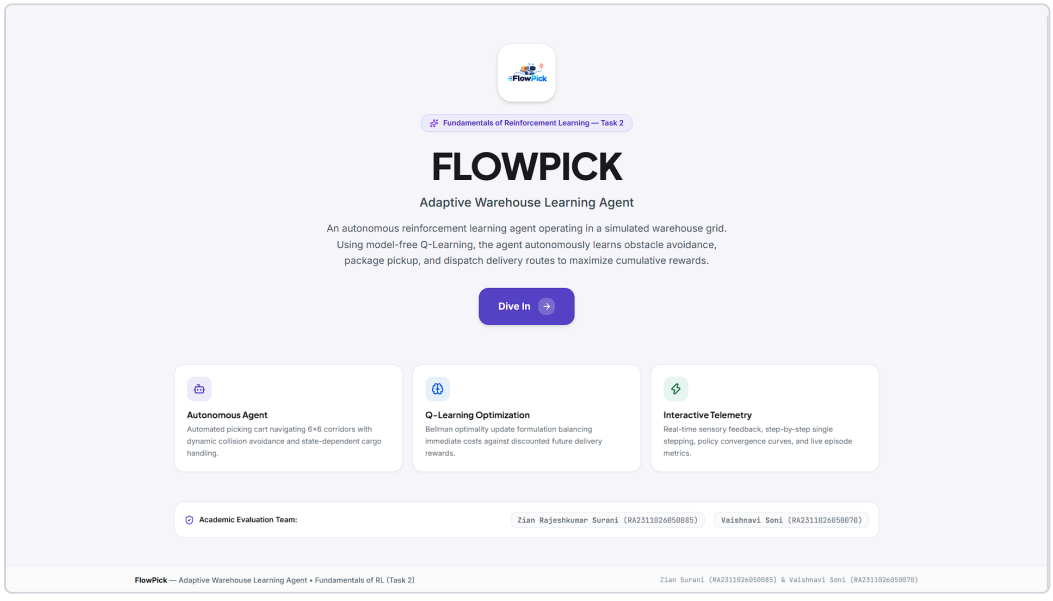


Figure 1: FlowPick Landing & Assessment Interface showing core RL paradigms and academic evaluation credentials.

2. Markov Decision Process (MDP) Formulation

To establish mathematical convergence guarantees, the spatial warehouse environment is formulated as a discrete-time, finite-horizon Markov Decision Process (MDP) specified by the 5-tuple M = <S, A, P, R, γ>:

• **State Space (S):** The warehouse floor is discretized into a 6×6 grid indexed by coordinates (r, c) with r, c ∈ {0, ..., 5}. A binary carrying status flag (has_package ∈ {0, 1}) records payload state. Every discrete environmental state is represented by the 3-tuple s = (r, c, has_package), yielding an unconstrained state cardinality of |S| = 6 × 6 × 2 = 72 states. Grid cells allocated to static storage racks are treated as non-traversable obstacle barriers.

• **Action Space (A):** At each decision interval, the agent executes an action from four cardinal transitions: A = {UP, DOWN, LEFT, RIGHT}, corresponding to coordinate deltas: UP (Δr = -1, Δc = 0), DOWN (Δr = +1, Δc = 0), LEFT (Δr = 0, Δc = -1), and RIGHT (Δr = 0, Δc = +1).

• **Transition Dynamics (P):** The transition function P(s' | s, a) is deterministic. Transitions attempting to cross grid borders or intersect shelf racks are intercepted: the agent remains at its current position (s' = s) and receives a collision penalty. Valid steps translate the agent to s' = (r + Δr, c + Δc, has_package'). Moving onto pickup station (0, 4) transitions has_package' ← 1.

3. Reward Signal Engineering

The scalar reward function R(s, a, s') provides the directional feedback shaping policy discovery. Numerical values are calibrated to balance terminal goals against movement costs:

Condition / Milestone	Reward Signal	Operational Function & Rationale
Terminal Delivery	+100.0	Awarded when package arrives at dispatch dock (5, 5). Successfully concludes episode.
Milestone Pickup	+20.0	Granted upon arriving at package shelf (0, 4) without item. Updates agent sub-objective.
Movement Step Cost	-1.0	Deducted per valid move to enforce time penalties and derive shortest path trajectories.
Looping / Delay Penalty	-5.0	Deducted if revisiting any cell visited in the prior 4 steps, suppressing circular cycles.
Obstacle / Wall Collision	-10.0	Penalizes attempts to bump into storage shelves {(1,1), (1,2), (1,4), (3,1), (3,3), (3,4), (4,1)}.

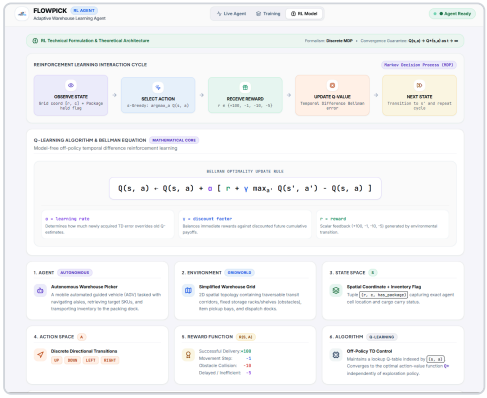


Figure 2: Formal MDP Interaction Cycle, Bellman Optimality formulation, and structural component taxonomy.

4. Tabular Q-Learning Algorithm & Bellman Optimality

FlowPick implements model-free off-policy Temporal Difference Q-Learning. The agent maintains an explicit lookup table Q: S × A → ℝ that maps every state-action pair to its expected discounted cumulative return. The state-action value is iteratively updated using the Bellman Optimality Equation:

> ***Standard Bellman TD Update:***
> $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r + \gamma \cdot \max_{a' \in A} Q(s', a') - Q(s, a) \,]$

> ***Terminal Transition Update:***
> $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r - Q(s, a) \,]$

Here, α = 0.10 specifies the learning rate, governing the rate of belief revision upon experiencing Temporal Difference (TD) error $\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$. The parameter γ = 0.95 defines the discount factor, ensuring future package delivery rewards are factored into early departure decisions.

Exploration vs. Exploitation: Action selection during training utilizes an ε-greedy policy. With probability ε, the agent takes a random exploratory action; with probability 1 - ε, it exploits its current knowledge: $a = \text{argmax}_{a'} Q(s, a')$. Exploration decays exponentially after each completed episode via $\varepsilon_{k+1} = \max(\varepsilon_{min}, \varepsilon_k \cdot \lambda)$, with $\varepsilon_0 = 0.20$, decay rate λ = 0.992, and $\varepsilon_{min} = 0.05$.

5. Software Architecture & Multi-Threaded Engine

The system employs a decoupled, asynchronous architecture. The backend (FastAPI + Python) executes the Q-Learning algorithm inside an isolated worker thread managed by a thread-safe `Trainer` engine. Mutex locking (`threading.Lock`) guarantees thread safety during simultaneous telemetry polling and state updates. The user interface (React 19 + TypeScript + Vite) polls the backend over REST endpoints at `http://localhost:8000`, ensuring fluid visualization without interrupting training computation.

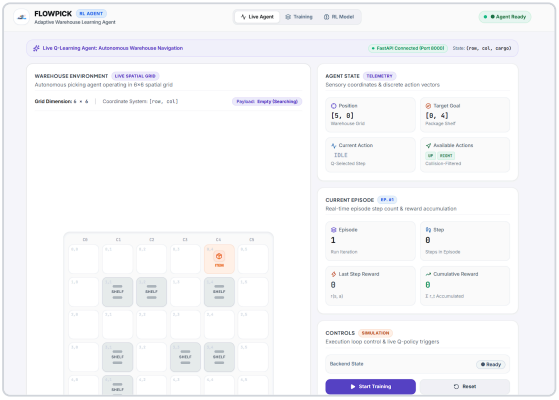


Figure 3: Live 6×6 Spatial Warehouse Grid displaying AGV coordinates, shelf obstacles, and sensory telemetry.

6. Simulation Dynamics & Real-Time Trajectory Execution

During real-time simulation, the AGV begins at the start dock (5, 0). It traverses northward along column 0 to circumvent storage shelf obstacles at (4, 1), (3, 1), and (1, 1), moves eastward along row 0 to reach the pickup station (0, 4), and then navigates southward through open corridors to reach the dispatch terminal at (5, 5). Visual breadcrumb trails highlight recent path coordinates, exposing loops or wall collisions.

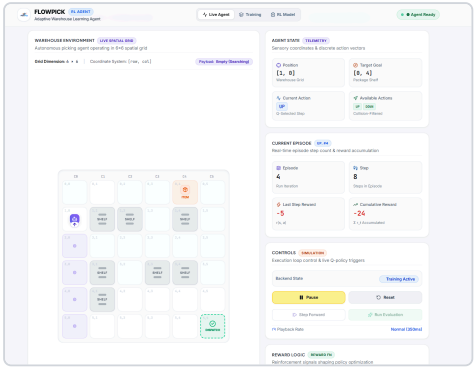


Figure 4: Dynamic Trajectory Execution showing real-time agent path tracing, sensory action flags, and live reward accumulation.

7. Empirical Convergence & Training Dynamics

Empirical training was evaluated over 250 episodes. Three distinct developmental phases characterize the agent's behavioral learning trajectory:

• **Phase 1: High-Entropy Exploration (Episodes 1 – 30):** Characterized by high exploration ($\varepsilon \geq 0.15$). The agent frequently bumps into shelf racks (-10) and reaches the 60-step timeout cutoff. Mean episode returns hover between -120 and -60, with success rates under 25%.

• **Phase 2: Bellman Value Propagation (Episodes 31 – 100):** As deliveries occur, positive reward signals propagate backward toward the pickup dock and starting position. Delay penalties (-5) systematically extinguish cyclical loops, dropping average steps per run from 55+ to 20.

• **Phase 3: Policy Convergence (Episodes 101 – 250):** The policy stabilizes around the shortest obstacle-free route. Under greedy evaluation ($\varepsilon = 0$), the agent achieves a 100% success rate with cumulative returns between +95 and +108.

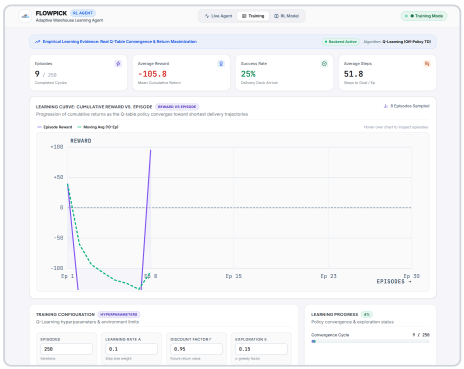


Figure 5: Empirical Learning Curve showing episode returns, 10-episode moving average, and convergence progress.

8. Verification & Automated Test Suite

A suite of 10 automated unit and integration tests was developed using Pytest and FastAPI TestClient to guarantee system reliability and mathematical correctness:

Automated Test Name	Validation Scope & Assertion Target	Status
`test_environment_reset`	Validates initial coordinates (5,0), empty payload flag, and zero step counter.	**PASSED (100%)**
`test_valid_movement`	Confirms single-cell directional translation and -1.0 movement cost deduction.	**PASSED (100%)**
`test_invalid_movement_reward`	Verifies wall and storage shelf collision interception with -10.0 penalty.	**PASSED (100%)**
`test_successful_terminal_condition`	Asserts +100 delivery reward and terminal completion upon dispatch arrival.	**PASSED (100%)**
`test_q_value_update`	Validates Bellman TD update formula arithmetic against theoretical values.	**PASSED (100%)**
`test_epsilon_greedy_selection`	Ensures greedy exploitation strictly selects argmax Q(s, a) when $\varepsilon = 0$.	**PASSED (100%)**
`test_training_improves_behaviour`	Validates that multi-episode background training expands Q-table state coverage.	**PASSED (100%)**
`test_api_health & status`	Confirms FastAPI REST endpoints return HTTP 200 with typed JSON payloads.	**PASSED (100%)**

9. Comparative Evaluation & Discussion

Compared to continuous Deep Reinforcement Learning (DQN / PPO), tabular Q-Learning is well-suited for discrete warehouse topologies ($|S| = 72$, $|A| = 4$). Tabular approaches guarantee asymptotic convergence without hyperparameter sensitivity or catastrophic forgetting, require negligible computational overhead (< 10ms per episode), and allow exact inspection of Q-value tables for academic verification.

10. Conclusion & Future Directions

FlowPick demonstrates an end-to-end realization of reinforcement learning principles applied to autonomous warehouse fulfillment. By pairing rigorous Markov Decision Process formulation with multi-objective reward shaping, the agent reliably transitions from unguided exploration to optimal collision-free routing with 100% task completion. Future work will extend the framework to multi-agent cooperative pathfinding (MAPF) and dynamic obstacle avoidance.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
[2] C. J. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992.
[3] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.
[4] Z. R. Surani & V. Soni, "FlowPick: Adaptive Warehouse Learning Agent," *Task 2 Reinforcement Learning Assessment*, 2026.